

建構垃圾留言機器學習模型

來源：<https://codelabs.developers.google.com/tflite-nlp-model?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解我們如何建立可過濾其他留言中垃圾內容的機器學習模型。

目錄

1. [1. 事前準備](#)
2. [2. 安裝 TensorFlow Lite 模型製作工具](#)
3. [3. 匯入程式碼](#)
4. [4. 下載資料](#)
5. [5. 預先學習的嵌入](#)
6. [6. 使用資料載入器](#)
7. [7. 建構模型](#)
8. [8. 匯出模型](#)
9. [9. 恭喜](#)

步驟 1 · 約 2 分鐘

1. 事前準備

在本程式碼實驗室中，您將檢閱使用 TensorFlow 和 TensorFlow Lite Model Maker 建立的程式碼，並使用以垃圾留言為基礎的資料集建立模型。原始資料可在 Kaggle 取得。並匯整為單一 CSV 檔案，移除損毀的文字、標記、重複的字詞等內容，這樣一來，您就能更專注於模型，而非文字。

您要查看的程式碼已在此提供，但強烈建議您在 [Google Colab](#) 中一併查看程式碼。

必要條件

- 本程式碼研究室適用於剛接觸機器學習的資深開發人員。
- 這個程式碼研究室是「開始使用行動裝置的文字分類功能」路徑的一部分。如果尚未完成先前的活動，請立即停止並完成。

課程內容

- 如何使用 Google Colab 安裝 TensorFlow Lite Model Maker
- 如何將雲端伺服器中的資料下載到裝置
- 如何使用資料載入器
- 如何建構模型

軟硬體需求

- 存取 [Google Colab](#)
-

步驟 2 · 約 1 分鐘

2. 安裝 TensorFlow Lite 模型製作工具

開啟 Colab。筆記本中的第一個儲存格會為您安裝 TensorFlow Lite Model Maker：

```
!pip install -q tf-lite-model-maker
```

完成後，請前往下一個儲存格。

步驟 3 · 約 1 分鐘

3. 匯入程式碼

下一個儲存格包含筆記本中程式碼需要使用的匯入項目：



```
import numpy as np
import os
from tflite_model_maker import configs
from tflite_model_maker import ExportFormat
from tflite_model_maker import model_spec
from tflite_model_maker import text_classifier
from tflite_model_maker.text_classifier import DataLoader

import tensorflow as tf
assert tf.__version__.startswith('2')
tf.get_logger().setLevel('ERROR')
```

這也會檢查您是否執行 TensorFlow 2.x，這是使用 Model Maker 的必要條件。

步驟 4 · 約 2 分鐘

4. 下載資料

接著，請將資料從 Cloud 伺服器下載到裝置，並將 `data_file` 設為指向本機檔案：

```
data_file = tf.keras.utils.get_file(fname='comment-spam.csv',  
    origin='https://storage.googleapis.com/laurencemoroney-  
blog.appspot.com/lmblog_comments.csv',  
    extract=False)
```

Model Maker 可從這類簡單的 CSV 檔案訓練模型。您只需指定哪些資料欄包含文字，哪些資料欄包含標籤。本程式碼研究室稍後會說明操作方式。

5. 預先學習的嵌入

一般來說，使用 Model Maker 時，您不會從頭開始建構模型，您可以使用現有模型，並根據需求自訂。

對於這類語言模型，這項程序會使用預先學習的嵌入。嵌入背後的概念是將字詞轉換為數字，並為整體語料庫中的每個字詞指定一個數字。嵌入是向量，用於判斷字詞的情緒，方法是為字詞建立「方向」。舉例來說，如果某些字詞經常出現在垃圾留言中，這些字詞的向量就會指向類似的方向，而不會出現在垃圾留言中的字詞則會指向相反方向。

使用預先學習的嵌入，即可從已從大量文字中學習情緒的字詞語料庫或集合開始。相較於從頭開始，這樣做能更快找到解決方案。

Model Maker 提供多種預先學習的嵌入，但最簡單快速的入門方式是使用 `average_word_vec`。

程式碼如下：

```
spec = model_spec.get('average_word_vec')
spec.num_words = 2000
spec.seq_len = 20
spec.wordvec_dim = 7
```

num_words 參數

您也可以指定模型要使用的字數。

您可能會認為「越多越好」，但一般來說，根據每個字詞的使用頻率，會有適當的數量。如果使用整個語料庫中的每個字詞，模型可能會嘗試學習並建立只使用一次的字詞方向。在任何文字語料庫中，許多字詞只會使用一到兩次，通常不值得在模型中使用，因為這些字詞對整體情緒的影響微乎其微。您可以使用 `num_words` 參數，依所需字數調整模型。

這裡的數字越小，模型就越小且越快，但由於可辨識的字詞較少，準確度可能會降低。這裡的數字越大，模型就越大，速度也越慢。找出最佳平衡點是關鍵！

wordvec_dim 參數

`wordvec_dim` 參數是指您要用於每個字詞向量的維度數量。根據研究結果，經驗法則是字數的四次方根。舉例來說，如果您使用 2000 字，建議從 7 開始。如果變更使用的字數，也可以變更這項設定。

seq_len 參數

模型通常對輸入值非常嚴格。以語言模型來說，這表示語言模型可以分類特定靜態長度的句子。這取決於 `seq_len` 參數，也就是序列長度。

將字詞轉換為數字 (或權杖) 後，句子就會變成一連串的權杖。因此，模型會接受訓練 (在本例中)，分類及辨識含有 20 個符記的句子。如果句子長度超過此限制，系統會截斷句子。如果較短，系統會填補空白。您會在用於此用途的語料庫中看到專屬的 `<PAD>` 符記。

6. 使用資料載入器

您先前已下載 CSV 檔案。現在，請使用資料載入器，將這個資料集轉換為模型可辨識的訓練資料：

```
data = DataLoader.from_csv(  
    filename=data_file,  
    text_column='commenttext',  
    label_column='spam',  
    model_spec=spec,  
    delimiter=',',  
    shuffle=True,  
    is_training=True)  
  
train_data, test_data = data.split(0.9)
```

如果您在編輯器中開啟 CSV 檔案，會發現每一行只有兩個值，而檔案第一行則以文字說明這些值。通常每個項目都會視為一欄。

您會看到第一欄的描述元是 `commenttext`，且每行的第一個項目都是留言文字。同樣地，第二欄的描述元是 `spam`，您會看到每行第二個項目是 `True` 或 `False`，表示該文字是否視為垃圾留言。其他屬性會設定您先前建立的 `model_spec`，以及分隔符號字元，在本例中為半形逗號，因為檔案是以半形逗號分隔。您將使用這些資料訓練模型，因此 `is_Training` 會設為 `True`。

您需要保留部分資料，用於測試模型。分割資料，其中 90% 用於訓練，另外 10% 用於測試/評估。由於我們要這麼做，因此請務必隨機選擇測試資料，而非資料集「底部」的 10%，因此載入資料時請使用 `shuffle=True` 隨機選擇。

步驟 7 · 約 4 分鐘

7. 建構模型

下一個儲存格只是用來建構模型，而且只有一行程式碼：

```
# Build the model
model = text_classifier.create(train_data, model_spec=spec, epochs=50,
                              validation_data=test_data)
```

這會使用 Model Maker 建立文字分類器模型，並指定要使用的訓練資料 (如步驟 4 中設定)、模型規格 (如步驟 4 中設定) 和訓練週期數 (本例為 50)。

機器學習的基本原則是模式比對。一開始，系統會載入字詞的預先訓練權重，並嘗試將字詞分組，預測哪些字詞組合在一起時表示垃圾內容，哪些則不是。第一次的結果可能接近 50:50，因為模型才剛開始運作。

```
Epoch 1/50
28/28 [=====] - 1s 21ms/step - loss: 0.6823 - accuracy: 0.5766
Epoch 2/50
28/28 [=====] - 0s 6ms/step - loss: 0.6308 - accuracy: 0.7130
Epoch 3/50
28/28 [=====] - 0s 6ms/step - loss: 0.5839 - accuracy: 0.7199
```

接著，系統會評估結果、執行最佳化程式碼來調整預測，然後再次嘗試。這是紀元。因此，指定 epochs=50 時，系統會執行 50 次「迴圈」。

```
Epoch 48/50
28/28 [=====] - 0s 5ms/step - loss: 0.0740 - accuracy: 0.9868
Epoch 49/50
28/28 [=====] - 0s 5ms/step - loss: 0.0721 - accuracy: 0.9872
Epoch 50/50
28/28 [=====] - 0s 6ms/step - loss: 0.0710 - accuracy: 0.9900
```

當您達到第 50 個訓練週期時，模型回報的準確率會高出許多。本例中顯示 99%！

右側會顯示驗證準確率數據。這些準確度通常會比訓練準確度略低，因為這代表模型分類先前「未見過」資料的準確度。這項作業會使用先前預留的 10% 測試資料。

```
Epoch 48/50
28/28 [=====] - 0s 5ms/step - loss: 0.0740 - accuracy: 0.9868 - val_loss: 0.2125 - val_accuracy: 0.9479
Epoch 49/50
28/28 [=====] - 0s 5ms/step - loss: 0.0721 - accuracy: 0.9872 - val_loss: 0.2128 - val_accuracy: 0.9479
Epoch 50/50
28/28 [=====] - 0s 6ms/step - loss: 0.0710 - accuracy: 0.9900 - val_loss: 0.2160 - val_accuracy: 0.9479
```

8. 匯出模型

訓練完成後，即可匯出模型。

TensorFlow 會以自有格式訓練模型，但這類模型必須轉換為 TFLITE 格式，才能在行動應用程式中使用。Model Maker 會為您處理這項複雜作業。

只要匯出模型並指定目錄即可：

```
model.export(export_dir='/mm_spam')
```

該目錄中會顯示 `model.tflite` 檔案。那就趕快下載，您會在下一個程式碼研究室中用到這個檔案，並將其新增至 Android 應用程式！

iOS 注意事項

您剛匯出的 `.tflite` 模型適用於 Android，因為模型的中繼資料會內嵌在其中，而 Android Studio 可以讀取該中繼資料。

這項中繼資料非常重要，因為其中包含代表單字的符記字典，模型會辨識這些單字。還記得稍早學到的概念嗎？文字會變成權杖，而這些權杖會獲得情緒向量。行動應用程式需要知道這些權杖。舉例來說，如果「dog」已權杖化為 42，而使用者在句子中輸入「dog」，應用程式就必須將「dog」轉換為 42，模型才能理解。Android 開發人員可以使用「TensorFlow Lite Task Library」簡化這項作業，但 iOS 開發人員必須處理詞彙，因此需要提供詞彙。Model Maker 可以指定

`export_format` 參數，為您匯出這項資料。因此，如要取得模型的標籤和詞彙，可以使用下列程式碼：

```
model.export(export_dir='/mm_spam/',  
             export_format=[ExportFormat.LABEL, ExportFormat.VOCAB])
```

步驟 9 · 約 1 分鐘

9. 恭喜

本程式碼研究室帶您瞭解如何使用 Python 程式碼建構及匯出模型。完成後，您會取得 .tflite 檔案。

在下一個程式碼研究室中，您將瞭解如何編輯 Android 應用程式以使用這個模型，開始分類垃圾留言。
